SERVER ARCHITECTURE EXPLORATION

Signals

From single-box CLI to SaaS: a deployment survey of the Signals backend across private server, Docker containers, Kubernetes, and AWS - grounded in the repository as it exists today, with a staged adoption roadmap.

Prepared for internal architecture review - October 30, 2026

SURVEY

Contents

Executive summary 3

The options explored . . . 3

Bottom line . . . 3

Part I — What we are deploying: the backend today 4

1.1 System overview . . . 4

1.2 Process model: jobs, not services . . . 4

1.3 State and storage inventory . . . 5

1.4 Scheduling shape today . . . 5

1.5 Outbound egress map . . . 6

1.6 Browser and antibot footprint . . . 7

1.7 Portability findings (the punch list) . . . 7

1.8 What "deployment" therefore means . . . 8

Part II — The decision framework 8

2.1 Four axes, decided separately . . . 8

2.2 Requirements the code imposes . . . 9

2.3 Non-requirements (evidence-based) . . . 9

Part III — Option A: Docker containers on a single host 9

3.1 Why Docker is the right first move . . . 9

3.2 Image design . . . 10

3.3 Topology on one host . . . 10

3.4 Storage and backup . . . 11

3.5 Costs and limits . . . 11

3.6 When this option wins outright . . . 11

Part IV — Option B: Kubernetes 11

4.1 What Kubernetes would give Signals . . . 11

4.2 The honest mismatch . . . 12

4.3 If/when we adopt it: the correct shapes . . . 12

4.4 Verdict . . . 12

Part V — Option C: AWS cloud integration 13

5.1 Level 1 — Lift and shift (EC2 + EBS) . . . 13

5.2 Level 2 — Managed surroundings (the sweet spot) . . . 13

5.3 Level 3 — Managed data plane (the SaaS-grade step) . . . 14

5.4 Fargate vs. EC2 vs. EKS for the job fleet . . . 14

5.5 Scheduling on AWS . . . 14

5.6 Egress and IP posture on AWS (the underrated section) . . . 15
5.7 Indicative cost models (monthly, planning-grade) . . . 15
5.8 AWS-lite alternatives, stated plainly . . . 15
Part VI — SaaS vs. private deployment 16
6.1 What SaaS actually requires (and doesn't exist yet) . . . 16
6.2 Multi-tenancy: which model fits this codebase . . . 16
6.3 The scraping posture under SaaS — the real risk section . . . 16
6.4 SaaS reference architecture (target state) . . . 17
6.5 Private deployment reference (target state) . . . 17
6.6 Economics sanity check . . . 18
6.7 Decision matrix . . . 18
6.8 The staged path (no rewrite, no cliff) . . . 18
Part VII — Comparative evaluation 18
7.1 Master comparison . . . 18
7.2 Risk register (top items) . . . 19
7.3 Recommendation . . . 19
Part VIII — Cross-cutting concerns 20
8.1 SQLite → Postgres migration checklist (the SaaS gate) . . . 20
8.2 Config and secrets in deployment . . . 20
8.3 Observability . . . 20
8.4 CI/CD . . . 21
8.5 Backups and DR . . . 21
8.6 Container security hardening . . . 21
Part IX — Phased roadmap 21
Appendix A — Reference Dockerfile (multi-stage, prod target) 22
Appendix B — Reference docker-compose.yml 22
Appendix C — Kubernetes sketches (for the deferred phase) 23
Appendix D — AWS Level 2 reference (private-scale) 23
Appendix E — Source egress summary (deployment allowlist seed) 24
Appendix F — Assumptions behind the cost figures 24
Appendix G — Punch list traceability 24

Executive summary

Signals today is a **batch CLI job system, not a web service**. A full-repo survey found zero HTTP servers: no FastAPI, Flask, uvicorn, ASGI/WSGI, or raw `http.server` anywhere under `src/`, `tests/`, `scripts/`, or `tools/`. The runtime is one Python 3.12 process at a time — `seed → resolve → collect → score → brief → export` — writing to a local SQLite database (WAL mode, 592 MB), a content-addressed gzip raw store (6,248 files, 174 MB), and a tree of markdown/JSON artifacts under `data/`. Concurrency is one `ThreadPoolExecutor(6)` for HTTP fan-out; everything else, including the browser tier, is serial. The only daemon shape that exists, `signals watch`, is an in-process `while True` loop whose own docstring recommends OS cron as the production driver.

That fact reframes the whole question. "Deploying the backend" does not mean shipping a web tier — there isn't one. It means three different things, and this document treats each seriously:

1. **Running the job system reliably somewhere** (scheduled collection, scoring, briefing, delivery) — a *packaging and scheduling* problem.
2. **Operating it privately vs. offering it as a product** — a *hosting model and business* problem.
3. **Adding the control plane a SaaS needs** (API, tenancy, auth, billing) — *net-new construction*, because none of it exists yet.

The options explored

Axis	Options examined	Verdict developed in
Packaging	Docker containers (single host)	Part III — **the foundation; do this first regardless of everything else**
Orchestration	Kubernetes (managed and self-hosted)	Part IV — **defer** until a multi-tenant worker fleet exists
Cloud	AWS at three adoption levels (EC2 lift-and-shift, managed surroundings, managed data plane)	Part V — **adopt incrementally**; a plain VPS beats AWS on price for the private case
Hosting model	Private server vs. SaaS	Part VI — **private now, SaaS-ready refactor later**; staged path defined

Bottom line

- **Docker is unconditionally worth it.** The image is the shared prerequisite of every other option (VPS, ECS, EKS, SaaS). Portability findings in §2.7 are the punch list.
- **Kubernetes is not the bottleneck or the win right now.** Signals' scaling constraint is its storage model — SQLite with WAL plus filesystem single-flight locks is single-writer by design. Kubernetes orchestrates containers; it cannot make two replicas share one SQLite file safely. Adopting K8s before swapping the database buys ceremony, not capability. Verdict: defer to the SaaS phase, and even then only for the worker fleet.
- **AWS is best adopted from the edges inward.** Secrets Manager, ECR, S3 backups, and EventBridge scheduling are low-risk, high-value first steps. RDS Postgres is the gate that unlocks real horizontal scale — and it is also the gate that makes SaaS credible.

- **Private deployment is the right 2026–2027 posture.** The SaaS control plane (API server, tenancy, billing) is net-new build work with real scraping-posture implications (§6.3). The staged path in §6.8 gets there without a rewrite.

Indicative monthly costs (planning-grade, USD, verify current pricing before committing): private VPS **$5–20**, AWS lift-and-shift **$15–40**, AWS with managed data plane **$80–200**, EKS floor **~$100+ before any workload**, SaaS seed **$200–500** plus per-tenant COGS of roughly **$10–30**.

Part I — What we are deploying: the backend today

Any architecture decision that ignores what the system actually does will optimize the wrong layer. This part inventories the runtime as it exists in the repository at commit `f6a8e62` (2026-10-30).

1.1 System overview

Signals is a sales-signal enrichment engine: it continuously collects buying signals about target accounts from ~30 self-built, zero-cost sources, resolves one identity per account, scores and tiers the signals per High Probability Prospecting (HPP), and maps each signal to a sales play. Outputs are cited dossiers per account, ranked briefs, digests, and exports.

The pipeline has six stages, driven by a Click CLI (`src/cli.py`, console script `signals`):

```
seed --> resolve --> collect --> score --> brief --> export
```

Two command families matter for deployment:

- `run` — full pipeline for a cohort (stage order implemented in `src/pipeline/orchestrator.py:854-878`).
- `intel` — the master intelligence flow: one account, every capability, one dossier (stages identity → collect → derive → score → package, `src/pipeline/intel.py:63`). Documented wall clock for a large account: ~30 minutes (`intel snowflake.com`, TODO.md).

1.2 Process model: jobs, not services

Property	Finding	Evidence
HTTP servers	**None.** No ASGI/WSGI/framework/raw-socket listener in the repo	full grep across `src/ tests/ scripts/ tools/`
Unit of work	CLI invocation (`python -m src.cli <command>`)	`pyproject.toml:45-46`
Only long-running shape	`watch` — in-process loop, `time.sleep(interval)` between ticks, CLI default 60 min	`src/pipeline/watch.py:56-135`
Recommended production driver	OS cron, per the module's own docstring	`watch.py:9-10`, `scheduler.py:1-11`
Concurrency	One `ThreadPoolExecutor`, `max_workers = config.http.max_workers` (default 6); browser-tier and network-capture tasks always serial	`runner.py:1143-1172`, `config/default.yaml:15`
Async	**None.** All HTTP is sync (`httpx.Client`, `curl_cffi`, sync Playwright/Patchright); `src/llm/implement.py` documents "plain loop, no threads, no asyncio"	`implement.py:10,711`

Property	Finding	Evidence
Cross-process exclusion	Single-flight lockfile `data/state/collect.lock` with PID-liveness checks	`watch.py:72-87`, `src/core/filelock.py`
Run audit	`runs` table + per-run rotating log sink `data/logs/<run_id>.log`	`src/core/runlog.py:47-90`

The durable scheduler (`src/pipeline/scheduler.py`) already implements what a deployment needs from a scheduler: per-source cadence from `config/sources.yaml`, ±5% jitter, exponential backoff (2^failures capped at 8x), state persisted in the `source_cursors` table, and stale-lock recovery after 24 h or a dead PID.

1.3 State and storage inventory

Everything mutable lives under `data/` (resolved against the process CWD) plus one external cache. Measured 2026-10-30:

Store	Path	Size	Character
Main database	`data/signals.db`	592 MB	SQLite, WAL mode, `foreign_keys=ON`, integrity check on every open, single connection serialized by an `RLock` (`check_same_thread=False`)
Raw document store	`data/raw/<xx>/<sha>.gz`	174 MB, 6,248 files	Content-addressed gzip, sha256-keyed, indexed by the `documents` table; pruned by age (`keep_raw_days: 400`)
Dossiers	`data/dossiers/`	123 MB	Intel output packages (markdown/JSON)
Logs	`data/logs/`	79 MB	Rotating 10 MB main sink + per-run files
Browser session	`data/state/session.json`	~20 KB	Playwright storage state
Antibot state	`data/antibot/*.json`, `data/cookies/`	small	Routing state machine, scoped cookie jars
Retrieval indexes	SQLite-internal	(in DB)	FTS5 shadow `chunks_fts` + `chunk_vectors` (float32 BLOBs) + `index_state` ledger — no separate vector DB
Model cache	user-home HF cache	~100 MB	MiniLM embedder + cross-encoder reranker, downloaded at first use from huggingface.co

Key structural facts (all in `src/core/db.py`): schema `SCHEMA_SQL` plus ordered additive migrations v1–v9; tables include `accounts` (~40 columns, the identity spine), `signals`, `documents`, `source_cursors`, `fetch_log`, `runs`, `jobs`, `job_snapshots`, `g2_reviews`, `calibration`, `play_outcomes`, and the retrieval set. `SIGNALS_DB_PATH` overrides the location (`config.py:376-378`).

Retention is built in: `prune` (defaults keep-days 90, stale-run-hours 6) deletes expired fetch_log/documents/runs rows and raw files, finalizes stuck `running` rows, then WAL-checkpoints and ANALYZEs (`cli.py:527-545`, `db.py:903-945`).

1.4 Scheduling shape today

There is no cron, APScheduler, or Celery code in the repo. The two mechanisms are:

1. `watch` — a durable in-process loop calling `orch.collect(...)` per due adapter, then `orch.score(...)` and the watchlist check, guarded by the single-flight lockfile.
2. **The** `Scheduler` — the state machine behind `watch` that decides *what* is due; docstrings in both `watch.py` and `scheduler.py` explicitly recommend driving `collect` per-source from **OS cron** in production.

Implication for every option that follows: the deployment unit is *a job*, and the scheduler question (cron vs. loop vs. cloud scheduler) is orthogonal to the packaging question (how the job is shipped).

1.5 Outbound egress map

The system is egress-heavy by design — ~30 sources, all self-built, zero-cost. Grouped from `config/sources.yaml` and the identity/xray modules:

Category	Hosts (representative)	Notes
Regulatory	`data.sec.gov`, `federalregister.gov`, `usaspending.gov`, DOL WARN, `crt.sh`, `web.archive.org`	SEC requires a real contact email in the User-Agent or it 403s
ATS vendors	`boards-api.greenhouse.io`, lever, ashby, smartrecruiters, workable, recruitee, workday (0.5 rps), rippling, jobvite, breezy, teamtailor, generic careers sitemaps	Newest arm: server-rendered detail pages, 903/903 jobs with content in the sourcing spike
News/RSS	`news.google.com` RSS, company blog feeds	2 rps
Community	HN Algolia, GitHub API (60/hr anonymous, 5,000/hr with free `GITHUB_TOKEN`), Reddit (disabled)	
Marketplaces (opt-in)	g2.com (browser tier + DataDome), capterra.com (curl_cffi + Cloudflare), trustradius, softwareadvice, getapp	`enabled: false` by default; browser tier stays off until configured
Identity	Wikidata/Wikipedia APIs, Google Knowledge Graph (service-account auth), DuckDuckGo HTML SERP (probe-gated, documented robots exception)	
X-ray (manual only)	`lite.duckduckgo.com` or Google SERP via browser-harvested cookies	Never scheduled; 20 s pacing between fetches
LLM (opt-in)	`openrouter.ai`, `api.typesafe.ai`	Off unless `decide.yaml mode != off`, `--with-llm`, keys, and the data-exit consent string are all present
Model CDN	`huggingface.co`	First-use downloads of the two quantized MiniLM models
Delivery	SMTP host, alert webhook URLs	STARTTLS on 587; HMAC-signed webhook payloads (legacy/hub/stripe schemes)
Captcha solvers (opt-in)	`api.2captcha.com`, `api.capsolver.com`	Only when CF/DataDome bypass configured; DataDome path also requires a residential proxy

Everything is polite by construction: robots.txt honored by default (two documented, probe-gated exceptions), rate limits are floors not targets, real contact address in the UA. This posture matters enormously when we get to SaaS (§6.3).

1.6 Browser and antibot footprint

- **Stock Playwright** (`src/core/browser.py`) and **Patchright** (patched Chromium, `--disable-blink-features=AutomationControlled --no-sandbox`, behavioral warm-up) drive the browser tier. Headless by default (`browser.enabled: false` gates the whole tier off until enabled).
- **Headed mode is a real requirement on some paths**: `PatchrightGhost` forces `headless=False` ("Headed mode is what clears DataDome", `src/antibot/python/ghost.py:62-88`), `g2-selfcheck` forces headed, and the Cloudflare tier-4 solver can flip headed via `CLOUDFLARE_HEADED_FALLBACK`. On Linux containers this implies **Xvfb** whenever those paths are enabled.
- **curl_cffi** (`impersonate="chrome"`) is the TLS-fingerprint tier between the plain fetcher and the browser — used by the DDG identity resolver, xray fetches, and Capterra.
- **The native engine** (`src/antibot/`, Rust PyO3 with vendored BoringSSL) is optional at runtime: without it, `SignalsTransport` degrades to curl_cffi and `doctor` reports a WARN, not a FAIL. The build tree is 566 MB and its documented build procedure is Windows-only (MSVC/vcvars64/NASM/Go). CI never builds it. **A Linux build recipe does not exist in-repo** — the container options below assume the curl_cffi fallback unless/until that recipe is written.

1.7 Portability findings (the punch list)

These are the concrete facts any packaging decision must handle. Each is citable and testable.

#	Finding	Why it matters for deployment	Fix difficulty
P1	All state paths are CWD-relative (`data/...`), and `runlog` re-reads `config/default.yaml` from CWD (`runlog.py:31`); config YAMLs themselves are package-relative (`config.py:257`)	Container must pin `WORKDIR` and mount one volume at the right place; `init` materializes the tree (CI's install-smoke proves Linux/CWD independence)	Trivial
P2	SQLite WAL + `O_EXCL`/PID lockfiles assume coherent local filesystem	The DB cannot live on NFS/EFS; single-writer constraint is structural, not incidental	Structural (drives Part IV/V decisions)
P3	Rate limits are in-process token buckets; single-flight is per-filesystem	Two replicas would double-hit sources and violate the politeness posture — horizontal scaling needs external coordination	Medium
P4	Headed-browser paths (ghost, g2-selfcheck, CF headed fallback) need a display	Xvfb in Linux containers if marketplace sources or headed fallback are enabled	Easy
P5	Sibling-repo coupling: reads `../Linkedin/data/linkedin.db` and `../repvue-scraper/data/repvue.db`; `deepen` shells out to `../Linkedin` with a **hardcoded Windows venv path** (`.venv/Scripts/python.exe`, `collector.py:91`)	Breaks on Linux without edits; silently no-ops when checkouts are absent	Easy–Medium
P6	Model downloads from huggingface.co at first use, cached in user home	Needs egress at runtime or a pre-baked cache volume/layer	Easy
P7	pytest 8.3.4 and respx are core (non-optional) dependencies	Production image installs test tooling unless a prod target strips them	Trivial

#	Finding	Why it matters for deployment	Fix difficulty
P8	Browsers are not auto-installed by any script/CI (`playwright install chromium` is a manual step)	Image build must install chromium + patchright chromium explicitly	Trivial
P9	No `/healthz` or metrics endpoint exists (there is no server to expose one)	Observability in deployment = logs + `runs` table + `status`/`doctor` commands, until a control plane exists	Medium (net-new)
P10	Windows-only native-engine build docs; Linux build recipe absent	Container images use the curl_cffi fallback (fully functional; `doctor` WARN)	Deferred
P11	PID-liveness check uses kernel32 via ctypes when `os.name == "nt"` (`filelock.py:42-52`)	Already has a non-Windows branch; CI matrix proves it	None

1.8 What "deployment" therefore means

For Signals, a deployment is the composition of **three units** plus one future one:

1. **A scheduler** — decides when sources are due (cron today; the `Scheduler` state machine provides the logic).
2. **A job runner** — executes CLI invocations (`collect`, `intel`, `score`, `index`, `digest`, `prune`, ...) against one coherent `data/` tree.
3. **Delivery** — already in-band (SMTP + signed webhooks + file destinations); needs no dedicated infrastructure, only egress and secrets.
4. *(Future)* **A control plane** — API, tenancy, auth, billing: net-new, exists nowhere in the repo today, and is the real cost of "SaaS."

Every option in Parts III–VI is a different answer to "where do 1–3 run, and who operates them" — plus "when do we pay for 4."

Part II — The decision framework

2.1 Four axes, decided separately

Conflating these four questions is the classic mistake. They have different owners, different costs, and different reversibility:

Axis	Question	Reversibility
Hosting model	Who runs it — you, privately, or tenants, as a product?	Low (business commitment, compliance posture)
Packaging	How does the code ship — bare host, container image?	High (images are portable everywhere)
Orchestration	What schedules and supervises the containers — cron, compose, K8s, a cloud scheduler?	Medium
Cloud	Which managed services absorb undifferentiated work — backups, secrets, DB, queue?	Medium-high, per service

Docker first is not a slogan; it is the observation that the container image is the artifact every other option consumes. Kubernetes without Docker images is impossible; AWS Fargate without an image is impossible; a

SaaS worker fleet is a fleet of images. Private bare-metal is the only option that skips it — and even there, compose is the cheapest insurance against the dev-box drift this repo already knows (Windows-only build docs, hardcoded venv paths).

2.2 Requirements the code imposes

Derived directly from Part I:

- **R1.** One coherent, locally-coherent `data/` tree per writer (P1, P2).
- **R2.** At most one collector writer per data tree at a time (P2, P3).
- **R3.** Egress to ~30 host families, with per-host rate floors honored; some paths additionally need residential-proxy egress (DataDome) and captcha-solver APIs.
- **R4.** Optional headed-browser capability (Xvfb on Linux) when marketplace sources are enabled.
- **R5.** Secrets via env indirection — the code never stores credentials, it names the env vars that hold them (`user_env`, `secret_env`, `SIGNALS_SMTP_PASS_ENV`). Any deployment platform must inject those env vars.
- **R6.** Scheduled cadence with jitter and backoff — already implemented in-process; the platform only needs to *invoke* jobs.
- **R7.** Modest resources: 6 HTTP workers + occasional Chromium + onnxruntime (2 intra-op threads); the whole data tree is under 1 GB today.
- **R8.** Runtime model fetch (or pre-baked model cache) from huggingface.co.

2.3 Non-requirements (evidence-based)

Just as important — these rule out entire categories of ceremony:

- **No web tier to scale.** There is no latency SLO, no connection pool, no health endpoint. Autoscaling HTTP frontends, service meshes, and ingress controllers solve problems Signals does not have.
- **No zero-downtime requirement.** A missed collection tick is recovered by the scheduler's backoff and cursors; deploys can happen between ticks.
- **No multi-region need.** Sources are global but the politeness posture (rate floors, contact address, IP reputation) favors *fewer, stabler* egress identities, not more.
- **No team-scale collaboration pressure.** One operator today; K8s' operational payoff starts when a team shares an environment.

Part III — Option A: Docker containers on a single host

3.1 Why Docker is the right first move

1. **Parity with CI.** GitHub Actions already runs the offline suite on ubuntu-latest and proves `pip install -e .` + `init` work in a clean `/tmp/smoke` dir. The Linux runtime story is validated continuously; an image freezes it.
2. **The punch list has a single choke point.** P1 (CWD-relative state) is solved by `WORKDIR /work` + one mounted volume. P4/P6/P7/P8 are one-line Dockerfile concerns. P5 needs a small code fix regardless.

3. **Reversibility.** The image is consumed identically by a VPS, an ECS task, an EKS pod, or a future SaaS worker. Nothing about choosing Docker forecloses anything else.
4. **Backup discipline.** One volume to back up; the DB gains a coherent `sqlite3 .backup` / restic story.

3.2 Image design

A multi-stage build with two targets:

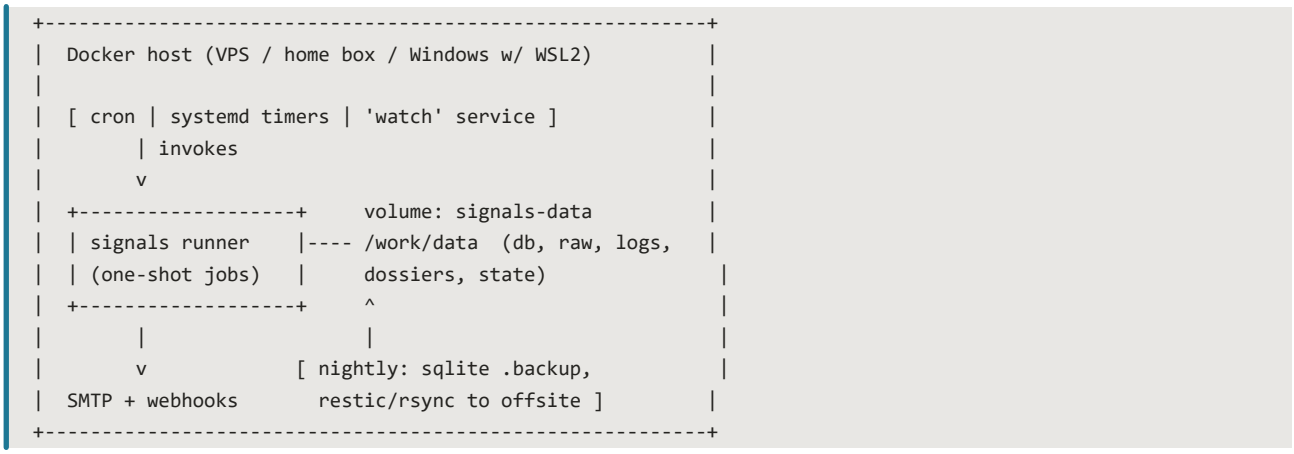
- `signals-base` — `python:3.12-slim`; system deps: `ca-certificates`, `curl`/`wget`, `tini` as PID 1; `pip install -e .` plus extras (`[rerank]` for the retrieval leg; optionally `[gkg]`); strip pytest/respx in the prod target (P7) by installing from an explicitly split dependency set, or accept them and keep the image honest.
- `signals-browser` — adds chromium system libraries, `playwright install chromium`, `patchright install chromium` (P8), and (optional variant) `xvfb` for the headed tier (P4). Browser tier stays off by config default; the capability ships disabled.

Practical points:

- **Models**: pre-bake the two pinned MiniLM files into an image layer (or an init container/job that runs the embedder once against a volume) so runtime never needs huggingface.co (P6).
- **Antibot engine**: excluded — curl_cffi fallback is the documented degradation and `doctor` treats its absence as WARN (P10). A Linux build recipe is future work, gated on whether the fallback's coverage is exceeded in practice.
- **Sibling repos** (P5): containerize by *absence* — the Linkedin/repvue integrations silently no-op when their checkouts are missing. If they are wanted, mount them read-only and fix the hardcoded `.venv/Scripts/python.exe` to respect `sys.executable` or an env var (a five-line change in `src/sources/linkedin_db/collector.py`).
- **Non-root user**, read-only rootfs, `tini` for signal handling, `SIGNALS_CONTACT_EMAIL` and tokens injected via env only (R5).

A complete reference Dockerfile is Appendix A.

3.3 Topology on one host



Three implementation choices, in order of preference:

1. **Cron invokes one-shot containers** (`docker compose run --rm signals collect`) — matches the repo's own recommendation (watch.py docstring), keeps the runtime stateless between ticks, and makes every run auditable in `docker/runs` history. Single-flight is guaranteed by "one host, sequential cron" plus the lockfile as backstop.
2. `watch` **as a restart-on-failure service** — acceptable, one less moving part, but conflates scheduling with execution inside one process; restart semantics blur run boundaries.
3. **K8s CronJobs on a single-node K3s** — possible now, sensible later (Part IV).

3.4 Storage and backup

- `signals-data` volume holds everything mutable (R1). The DB (592 MB) plus raw store (174 MB) fit comfortably on any host.
- Backups: nightly `sqlite3 signals.db ".backup ..."` (safe online backup, no WAL hazards) + `restic`/`rsync` of the volume to offsite storage (S3-compatible if AWS is adopted; otherwise any object store). `prune` already enforces retention inside the volume; backup rotation handles the outside.
- Do **not** mount the DB volume over network filesystems (P2). If two hosts must share read access later, that is the Postgres trigger, not an NFS argument.

3.5 Costs and limits

- **Cost**: effectively $0 on existing hardware; $5–20/month on a small VPS (Hetzner CX22/OVH/DigitalOcean class, 2 GB RAM) — enough for 6 workers + occasional headless Chromium (headed G2 solos want 4 GB).
- **Limits, accepted**: single host, no HA, manual rollouts, backups replace failover. For a personal/sales-team tool with recoverable ticks, these are *the right tradeoffs today*, and every one of them has a defined escalation path in later parts.

3.6 When this option wins outright

- One operator, one data tree, private use.
- Deploy cadence measured in days, SLA measured in "cron eventually ran."
- Zero appetite for cloud billing complexity.

This describes Signals today. Docker-compose-on-a-VPS is the honest 2026 answer, and it is *not* a dead end — it is phase 1 of every later option.

Part IV — Option B: Kubernetes

4.1 What Kubernetes would give Signals

- Declarative **CronJobs per source family** with `concurrencyPolicy: Forbid` — a clean mapping onto the scheduler's per-source cadence.
- **Jobs for one-shot flows** (`intel acme.com`, `index`, `digest`), with retries and history.

- **Self-healing and rollouts** for anything long-running; `kubectl`/GitOps workflow; namespaces as the seam for future tenant isolation.
- A real answer to "what runs this when I have ten tenants and forty heterogeneous jobs."

4.2 The honest mismatch

Kubernetes schedules *containers*; Signals' constraint is *state*.

- The single-flight lock is per-filesystem and the DB is SQLite-on-local-disk (P2, P3). Two replicas against one PVC: the second one double-hits sources and races WAL on a filesystem that may not honor POSIX locks as SQLite requires. Two replicas against two PVCs: two divergent data trees.
- So a K8s deployment of today's code is **exactly one writer** with extra steps: `Deployment replicas=1` with `strategy: Recreate`, or CronJobs serialized by `Forbid` + the lockfile backstop. That is a valid design (crash-only jobs, declarative history) — but it buys operational reach, not scale, and its price is the highest learning/billing curve of any option here.
- The actual unlock for horizontal scale is **swapping the storage model** (Postgres + a queue), which is a prerequisite for SaaS anyway (§6.2, §8.1) — *not* Kubernetes itself. K8s becomes valuable *after* that swap, when workers are stateless and replicas are safe.

4.3 If/when we adopt it: the correct shapes

- **Jobs, not daemons**: one CronJob per source cadence tier (hourly/daily/weekly), `concurrencyPolicy: Forbid`, `activeDeadlineSeconds` sized from documented run durations (~30 min worst-case intel).
- **Single-writer discipline** until Postgres: anything touching `data/` pins to `replicas: 1` + `ReadWriteOnce` PVC; read-only commands (accounts, signals, plays-report, status) can scale freely against a `ReadOnlyMany` mount — or simply read via the control-plane API once it exists.
- **Managed vs. self-hosted**: EKS carries a ~$73/month control-plane floor plus node costs — indefensible for one tenant; GKE Autopilot and AKS are similar. For private scale, **K3s** (or k0s) on a single VPS gives the API surface without the bill, and is a legitimate staging environment for the SaaS phase.
- A manifest sketch is Appendix C.

4.4 Verdict

Defer — with a precise trigger. Adopt K8s when *both* are true: (a) the storage model has been externalized (Postgres + queue, §8.1) so workers are stateless, and (b) a real fleet exists — multi-tenant SaaS data plane, or 3+ environments shared by more than one operator. Until then, every dollar and hour spent on K8s is spent on problems Signals does not have, and the Docker image work it displaces is the same work K8s would require anyway.

Part V — Option C: AWS cloud integration

AWS is not one decision; it is a ladder. Each rung is independently reversible and independently priced. Costs below are **indicative planning figures** (USD, late-2026 knowledge, us-east-1 class pricing) — re-verify against current AWS pricing before commitment.

5.1 Level 1 — Lift and shift (EC2 + EBS)

The Part III stack on a managed Linux box: EC2 (t4g.small/medium, ~$12–25/mo), EBS gp3 volume for `signals-data`, plain cron in-system. Amazon Linux 2023 or Ubuntu 24.04; Docker if desired.

- **Gains**: no 3 a.m. hardware worries; snapshots of EBS are one API call; the box is disposable, the volume is not.
- **Costs**: ~$15–40/mo all-in; plus either a public IP (free in the default v2 model) or a NAT decision (§5.6).
- **Honest comparison**: a $5–20 VPS does the same job. Pay the AWS premium only when the adjacency to other AWS services (below) is worth it.

5.2 Level 2 — Managed surroundings (the sweet spot)

Keep compute simple; absorb undifferentiated operations into managed services:

Signals concern	AWS service	Why it fits	Swap cost
Image registry	ECR	CI already on GitHub Actions; push tags from the existing workflow	Trivial
Secrets (solver keys, SMTP, webhooks, GKG JSON)	Secrets Manager (~$0.40/secret/mo) or SSM Parameter Store (free tier)	Matches R5's env-indirection pattern; resolve to env vars at task start	Trivial
Raw store / exports / backups	S3 (~$0.023/GB-mo; the whole tree is < 1 GB → cents)	Content-addressed sha256 keys map 1:1 to `data/raw/<xx>/<sha>.gz`; `RawStore.put/get/prune/stats` is the only seam to touch	Small, well-seamed
Scheduling	EventBridge Scheduler → ECS RunTask (or a cron container on one EC2)	Replaces cron with an audited, retry-capable scheduler; per-invocation pricing is negligible at this scale	Small
Compute	**ECS on Fargate** (~$0.04/vCPU-hr + ~$0.0045/GB-hr; spot ~70% off for batch)	One task definition per job type; no OS to patch; `runs` history stays in-app	Small
Delivery	SES ($0.10/1,000 emails) for digests; SNS/webhook egress for alerts	SMTP relay without running one; alert path already HMAC-signs	Optional
Logs/metrics	CloudWatch Logs (~$0.50/GB ingested) + `runs` table dashboards	Replaces tail-grepping `data/logs`	Optional
DB	**unchanged at this level** — SQLite stays on a Fargate ephemeral task + S3 snapshot restore, or on one EC2 with EBS	Fargate tasks are ephemeral: either re-attach an EFS (violates P2) or snapshot/restore DB to S3 per run; for the single-tenant case, one small EC2 with EBS is the pragmatic shape	See 5.3

The SQLite-on-AWS question. This is the crux of every AWS architecture:

- EFS + SQLite WAL is a known-bad combination (NFS locking semantics violate SQLite's assumptions — P2 again).
- Therefore: either stay **one EC2 + EBS** (Level 1 semantics with Level 2 services around it), or **Fargate + per-run DB snapshot to S3** (viable because runs are minutes and writes are bounded), or take the **RDS Postgres** step (Level 3).
- Recommendation: Fargate + snapshot/restore for the private case; RDS when the SaaS refactor begins.

A reference architecture for Level 2 is Appendix D.

5.3 Level 3 — Managed data plane (the SaaS-grade step)

Component	Swap	Effort
SQLite → **RDS Postgres** (db.t4g.small ~$25/mo single-AZ) or Aurora Serverless v2 for spiky SaaS loads	`src/core/db.py` is the single seam; dialect work: FTS5 → Postgres FTS (`tsvector`), `chunk_vectors` BLOB → `pgvector`, migrations v1–v9 → Alembic, drop the `RLock` serialization	Significant, one-time; checklist in §8.1
Raw store → **S3** (already staged at Level 2)	`rawstore.py` seam	Small
Cron → **SQS** + worker pool (~$0.40/M requests; first 1M free)	Scheduler enqueues due sources; stateless workers consume; replaces filesystem single-flight with queue semantics	Medium
Embeddings (optional)	pgvector already covers it; managed OpenSearch only if FTS+vector fusion outgrows Postgres	Deferred

Level 3 is precisely the storage refactor SaaS needs (§6.2). Building it once serves both.

5.4 Fargate vs. EC2 vs. EKS for the job fleet

Criterion	EC2 (self-managed cron)	ECS Fargate	EKS
Ops burden	You patch the box	None	Highest
Cost floor (private scale)	~$12–25/mo	~$5–15/mo for bursty batch tasks	~$73/mo + nodes
Batch-job fit	Good (it's cron)	Excellent (per-job tasks, spot)	Good (CronJobs)
Egress/IP story	Stable instance IP	NAT-gateway-dependent (§5.6)	Same as Fargate
Multi-tenant readiness	Low	Medium (task defs per tenant class)	High (namespaces/RBAC)
When it wins	Private, cost-minimal	Private→SaaS transition	SaaS fleet

5.5 Scheduling on AWS

- **EventBridge Scheduler → ECS RunTask** is the direct analog of cron-in-compose: one schedule per source tier, `retryPolicy`, dead-letter via CloudWatch alarm. Cheapest correct option; no Step Functions needed for flat schedules.

- **Step Functions** earns its keep only for the `intel` flow's intra-run orchestration (identity → collect → derive → score → package with per-stage retry) — nice-to-have; the in-process orchestrator already sequences these stages with its own run audit. Defer until SaaS demands per-stage visibility for tenants.

5.6 Egress and IP posture on AWS (the underrated section)

- Scraping is request-small, response-heavy: outbound bytes are minor, but **all of it transits a NAT gateway if tasks are private** — $32/mo baseline + $0.045/GB processed. For a low-volume crawler that is acceptable but silly; alternatives: public-subnet tasks with auto-assigned public IPs (free, but IP rotates per task — interacts with source rate limits and IP-reputation posture), or one EC2 runner with a stable Elastic IP.
- **IP stability is a politeness asset**: SEC EDGAR, GitHub, ATS vendors, and News RSS all reward consistent identities with stable rate floors. Rotating cloud IPs per task is anti-polite. A single stable egress (VPS or one EC2) matches the repo's zero-cost, well-behaved philosophy better than a churning fleet.
- DataDome-tier work requires a **residential proxy** anyway (config `datadome.residential_proxy`) — cloud egress is irrelevant for that path; the proxy provider is the identity.
- Keep the contact email honest and constant (SEC 403s otherwise; `config.py:273-283`).

5.7 Indicative cost models (monthly, planning-grade)

Scenario	Composition	Est. cost
Private VPS (baseline)	2 GB VPS + offsite object backup	**$5–20**
AWS Level 1	t4g.small EC2 + EBS + snapshots	**$15–40**
AWS Level 2 (Fargate batch)	Fargate spot tasks (~60 vCPU-hr) + ECR + Secrets + S3 + EventBridge + CloudWatch	**$25–70**
AWS Level 3 (+ RDS)	Above + db.t4g.small single-AZ + pgvector	**$55–95**
EKS anything	Control plane before a single job runs	**~$73+**
SaaS seed (10 pilot tenants)	Level 3 + control-plane ECS service + SQS + SES	**$200–500**

Reading: at private scale AWS is a convenience premium over a VPS; at SaaS scale the managed services (RDS, SQS, SES, Fargate spot) are what make a small team viable.

5.8 AWS-lite alternatives, stated plainly

The private case is price-sensitive and egress-heavy: a Hetzner/OVH class VPS delivers the same runtime for a third of the price, and Lightsail ($5–10) is AWS's own answer to that market. The rational AWS boundary for *private* deployment is Level 2's free-tier-friendly edges (SSM Parameter Store, ECR, S3 backups) even while compute stays on the cheapest capable box. This document's recommendation (§7.4) reflects exactly that split.

Part VI — SaaS vs. private deployment

6.1 What SaaS actually requires (and doesn't exist yet)

The SaaS delta is not infrastructure — it is a **control plane** that is nowhere in the repo. Building it means:

Capability	Today	To build
API server	None (no HTTP listener in repo)	FastAPI (or similar) service: authn/z, tenant CRUD, trigger runs, read briefs/dossiers/signals
Tenancy	Single-tenant by construction (one `data/` tree, one operator's account registry)	Tenant registry + isolation model (§6.2)
Billing/metering	None	Per-tenant usage metering (runs, documents, LLM spend, solver spend)
Onboarding	Manual seed/sweep by the operator	Self-serve: domain list import, ICP config, verification
Multi-tenant scheduling	`source_cursors` keyed `'global'` per source	Cursor keys per tenant; fairness across tenants
Status/SLA	`status`/`doctor` CLI, `runs` table	Status page, per-tenant health, alerting
Support surface	Direct	Tickets/SLAs — a *process* cost, not code

6.2 Multi-tenancy: which model fits this codebase

Model	How it maps to Signals	Pros	Cons
DB-per-tenant (SQLite)	Each tenant gets a `data/` tree + `signals.db` + cursors; workers already take a DB path via `SIGNALS_DB_PATH`	**Near-zero schema risk** — no tenant columns across ~40-column tables; blast radius per tenant is a file; matches the content-addressed raw store and dossier layout naturally; scheduler fans out per tenant	Fleet of files needs orchestration; cross-tenant analytics is hard; more moving parts at scale
Shared Postgres + `tenant_id`	Add tenant columns, row-level security, shared cursors/fetch_log	One system to back up; cross-tenant queries; the "real" SaaS shape	Invasive refactor (R2/P3 constraints move into schema); every table and query touched; RLS correctness burden
Hybrid (staged)	Start DB-per-tenant; migrate hot/large tenants into shared PG later	Revenue and experience learnings fund the refactor	Two models to support during transition

Recommendation: **hybrid, starting DB-per-tenant.** The codebase's existing seams (`SIGNALS_DB_PATH`, CWD-relative state, per-run audit) make the per-tenant model almost free; the shared-Postgres model is exactly the §8.1 refactor, adopted when scale (not ideology) demands it.

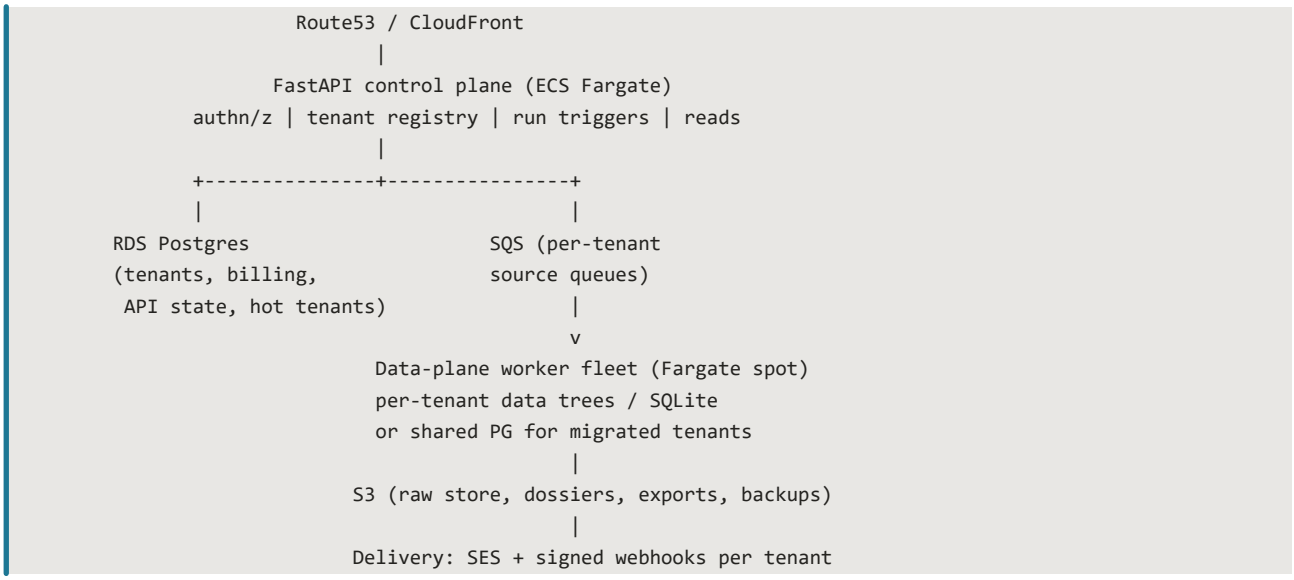
6.3 The scraping posture under SaaS — the real risk section

Signals' economics are built on **zero-cost, polite, self-built sources**. SaaS changes the surface, not the philosophy:

- **Concentrated egress**: today one operator's IP reputation is one operator's problem. A SaaS fleet scrapes from shared cloud IPs at the *sum* of all tenants' volumes — the fastest way to convert free sources into blocked sources. Mitigations: stable egress identities (§5.6), strict per-tenant rate budgets (the token buckets are in-process today — they must become per-tenant-aware before any multi-tenant run), and honoring per-host floors fleet-wide.
- **Compliance surface**: robots posture, SEC EDGAR fair use, and the two documented DDG exceptions were designed for one operator's manual/probe-gated behavior. Under SaaS, "who is the contact?" (`SIGNALS_CONTACT_EMAIL`) and "whose consent covers the LLM data-exit?" (`SIGNALS_DECIDE_DATA_EXIT`) become legal questions, not config questions.
- **Keyed-path costs stop being optional**: captcha-solver spend (2captcha/capsolver) and residential-proxy spend are per-tenant COGS the moment any tenant needs G2/DataDome-tier sources. These were "documented exceptions" in the zero-cost model; they are line items in a SaaS model.
- **Source ToS drift**: marketplace/gated sources can change terms faster than a single operator notices. A SaaS needs a source-compliance register and kill-switch per source per tenant (the `source_cursors.fail_count/error_class` machinery is a good seed for this).

None of this says *no*; it says **SaaS is a product decision with ongoing legal/ops tail, not a deployment setting.**

6.4 SaaS reference architecture (target state)



The control plane is stateless and boring (that is a feature); the data plane inherits everything from Part III's image. Nothing in this picture requires Kubernetes on day one; ECS suffices until fleet-shape complexity (§4.4's trigger) is real.

6.5 Private deployment reference (target state)

The Part III topology, hardened: compose on a VPS or home box; `watch`-or-cron per §3.3; nightly `sqlite3 .backup` + restic to S3/B2; Tailscale/WireGuard for access; `doctor` + `status` wired into a weekly email digest (the `digest` command already does delivery). Operator: you. SLA: "cron with receipts."

6.6 Economics sanity check

The README's stated thesis: commercial signal platforms are priced per seat and still need gluing together. At the COGS estimates above (compute + DB share + egress + metered solver/proxy only where enabled), a seat-priced offering at even a fraction of commercial per-seat prices carries healthy margin — *provided* the keyed paths (solvers, residential proxies, LLM spend) are metered per tenant rather than absorbed. The decision gate is not unit economics; it is §6.3's ongoing posture plus the net-new control-plane build (§6.1).

6.7 Decision matrix

If this is true...	Choose
It is your own prospecting weapon; nobody else needs an account	**Private** (Part III today; AWS Level 2 edges if you want offsite muscle)
A handful of trusted colleagues want in	**Private multi-tree** — DB-per-"tenant" without billing; same image
Strangers should be able to sign up and pay	**SaaS path** — begin the §8.1 storage refactor + control plane; start on ECS; revisit K8s at the §4.4 trigger
You want SaaS economics without SaaS obligations	**Hybrid**: private core + API-keyed read access to briefs/digests for a pilot cohort; validates demand before the control-plane investment

6.8 The staged path (no rewrite, no cliff)

1. **Stage 0 — private (now):** Docker image + compose + backups. All punch-list items fixed.
2. **Stage 1 — private, hardened:** offsite backups, monitoring digest, documented restore drill.
3. **Stage 2 — pilot:** second/third `data/` tree for trusted users on the same host; per-tree rate budgets exercised; sibling-repo fixes land upstream.
4. **Stage 3 — SaaS seed:** Postgres migration (§8.1) + minimal FastAPI control plane (auth, trigger, read) + SQS between scheduler and workers; deploy on ECS Fargate; waitlist onboarding.
5. **Stage 4 — product:** billing, tenant onboarding, per-tenant compliance register, K8s decision point per §4.4.

Each stage consumes the previous stage's artifacts unchanged; no stage is throwaway.

Part VII — Comparative evaluation

7.1 Master comparison

Criterion	Bare host + cron	Docker Compose (single host)	K8s (self-hosted K3s)	K8s managed (EKS)	ECS Fargate (AWS)
Time to first deploy	Hours	Days	Weeks	Weeks+	Days–weeks
Ops burden (steady state)	Medium (box drift)	**Low**	High	Highest	Low-medium
Cost floor, private scale	$0–5	$5–20	$10–25	~$100+	$25–70

Criterion	Bare host + cron	Docker Compose (single host)	K8s (self-hosted K3s)	K8s managed (EKS)	ECS Fargate (AWS)
Fits SQLite single-writer model	Yes	Yes	Awkward, but possible	Awkward, but possible	Yes (snapshot/restore)
Horizontal scale path	Manual	Manual (Postgres trigger)	Good, post-Postgres	Good, post-Postgres	Good, post-Postgres
Multi-tenant readiness	Low	Medium (per-tree)	High (namespaces)	High	Medium-high
Reversibility	n/a	**Total**	High	High	Medium
Team collaboration payoff	None	Low	Medium-high	High	Medium
Vendor lock-in	None	None	Low (K8s API)	Medium (IAM/EKS)	Medium (ECS API)

7.2 Risk register (top items)

Risk	Options most exposed	Mitigation
Source blocking from IP churn/reputation	Multi-replica anything; churning Fargate public IPs	Stable egress identity; per-host floors; residential proxy only for DataDome tier (§5.6, §6.3)
SQLite-on-network-storage corruption	Any design that puts the DB on EFS/NFS	Never do it; single-writer on local disk; Postgres when two writers are real (§8.1)
Double-hitting sources from two collectors	Any horizontal scale before the queue exists	Filesystem single-flight today; SQS + per-tenant cursors in SaaS phase
Secrets leakage into images/logs	All container options	Env indirection preserved; Secrets Manager/SSM; never bake `.env`; loguru sinks to stdout + rotated files, not to images
Headed-browser tier silently degraded in containers	Marketplace sources enabled	Xvfb variant image; `g2-selfcheck` in CI nightly lane as drift canary
Model-fetch flakiness at boot	All	Pre-baked model layer/volume (P6)
Cost surprises (NAT, EKS floor, CloudWatch)	AWS options	Budgets + alarms on day one; cost table §5.7 reviewed quarterly

7.3 Recommendation

1. **Do now:** the Docker image + compose stack (Part III), fixing punch-list items P1–P8 (P9–P11 as flagged). Deploy privately on the cheapest capable host.
2. **Adopt selectively:** AWS Level 2's edges that cost cents — ECR, SSM Parameter Store, S3 backups — even while compute stays on the VPS. Skip the NAT gateway for private scale.
3. **Defer, with triggers:** Kubernetes until §4.4's trigger (stateless workers + real fleet). RDS Postgres until Stage 3. Step Functions until per-stage tenant visibility is a requirement.
4. **Decide as a product question, not a technical one:** SaaS (Part VI) — when the answer is yes, Stages 3–4 are already mapped and every prior artifact is reused.

Part VIII — Cross-cutting concerns

8.1 SQLite → Postgres migration checklist (the SaaS gate)

The good news: every read/write already flows through the `Database` class (`src/core/db.py`) — a single seam. The checklist:

1. **Dialect**: migrate `SCHEMA_SQL` + migrations v1–v9 into Alembic; replace SQLite `UPSERT`/`PRAGMA` usages; parameter-style (`?` → `%s`).
2. **FTS5 → Postgres FTS**: `chunks_fts` shadow becomes a `tsvector` column + GIN index; `index --rebuild-fts` maps to vacuum/reindex.
3. **Vectors → pgvector**: `chunk_vectors` float32 BLOBs → `vector(384)`; cosine ops in SQL replace `fusion.py` in-memory math if desired (keep the Python fusion initially — correctness first).
4. **Concurrency model**: drop the `RLock`/single-connection serialization; adopt a pool (`psycopg_pool`); verify `HttpFetcher` logging path (the reason for `check_same_thread=False`).
5. **Single-flight → queue semantics**: `data/state/collect.lock` and scheduler cursors move to `SELECT ... FOR UPDATE SKIP LOCKED` or SQS visibility timeouts (P3 resolved structurally).
6. **Retention**: `prune` becomes a scheduled job against SQL partitions/cutoffs; WAL checkpoint/vacuum steps removed.
7. **Raw store**: `rawstore.py` → S3 keys (`s3://.../<xx>/<sha>.gz`); `documents.path` semantics updated; retrieval chunk reads follow.
8. **Verification**: the offline test lane (295 test files, network-blocked conftest) runs against Postgres in CI via a service container — the suite is the migration's safety net.

8.2 Config and secrets in deployment

The repo's env-indirection pattern (R5) is already container-correct. Rules:

- `.env` never enters an image; only `.env.example` is tracked.
- In AWS, Secrets Manager/SSM values are resolved into env vars at task start; the GKG service-account JSON mounts as a file (its path is the env var).
- All 13 `config/*.yaml` are static and trackable — bake into the image; the human-curated `config/lists/` mount as a volume (they are operator state, not code).

8.3 Observability

Today: loguru to stderr + rotating files, per-run sinks, `runs` table, `status`/`doctor`. In deployment:

- Containers log to **stdout** (Docker/K8s/CloudWatch-native); keep the file sinks only when running bare.
- `runs` table is the real telemetry — expose it via a weekly digest or (post-control-plane) a `/runs` API; add `doctor`-as-CronJob with webhook alerts on FAIL.
- Health checks: none exist (P9). For compose, a `doctor --no-network` healthcheck is honest and cheap. A `/healthz` appears naturally with the FastAPI control plane.

8.4 CI/CD

The existing workflow (offline lane on ubuntu+windows, install-smoke, nightly live-parity) needs three additions for any container future:

1. **Build and push** the image on tag (GHCR for private, ECR if AWS): lint stage exists (ruff, warn-only) — keep.
2. **Smoke the prod target**: `docker run --rm <image> doctor --no-network` + `init` in a scratch dir — the container analog of the existing install-smoke.
3. **Deploy step** per target: compose pull-up on the VPS (SSH action), `aws ecs update-service`/RunTask for AWS, `kubectl rollout` post-K8s.

8.5 Backups and DR

- **RPO 24 h, RTO 1 h** is proportionate for this system (ticks are recoverable; the irreplaceable asset is the DB + curated lists).
- Nightly: `sqlite3 .backup` (online, WAL-safe) → compress → offsite object storage; restic over the whole volume weekly.
- Quarterly: restore drill into a scratch tree + `doctor` + row-count parity check.
- Post-RDS: automated snapshots + the same drill.

8.6 Container security hardening

Non-root UID, read-only rootfs with `data/` the only writable mount, `tini` init, no secrets in layers, pinned base-image digest, egress allowlist where the platform permits (the §1.5 table is the allowlist), and Dependabot/`pip-audit` on the image's lockfile. The code's no-credential-storage design (R5) means a leaked image leaks nothing — keep that property sacred.

Part IX — Phased roadmap

Phase	Scope	Effort	Exit criteria
0 — Containerize	Punch-list fixes (P1–P8); multi-stage Dockerfile (base + browser variants); compose file; model pre-bake; `deepen` portability fix upstreamed	2–4 days	`docker run signals doctor --no-network` green; offline suite green in-container; `intel` run completes against a volume
1 — Private deploy	VPS or home box; cron (or `watch` service); nightly `.backup` + restic offsite; weekly `digest` email as heartbeat	1–2 days	Restore drill passes; two weeks of ticks with zero manual intervention
2 — Cloud edges (optional)	ECR mirror of the image; SSM Parameter Store for secrets; S3 backup target; CloudWatch alarm on failed ticks	1 day	Secrets out of the VPS; backups off-premises
3 — SaaS seed (gated on the §6.7 product decision)	Postgres migration per §8.1; FastAPI control plane (auth, tenant CRUD, trigger, read); SQS scheduler→worker; ECS Fargate deploy	3–6 weeks	Two pilot tenants isolated end-to-end; per-tenant rate budgets enforced
4 — Product	Billing + metering; onboarding; compliance register; K8s decision per §4.4 trigger	Ongoing	First paying tenant; fleet-shape complexity measured, not assumed

Explicit deferrals (with triggers): Kubernetes (§4.4), Step Functions (§5.5), managed OpenSearch (§5.3), Linux native-engine build (P10 — only if the curl_cffi fallback's coverage is exceeded), multi-writer SQLite on network storage (never).

Appendix A — Reference Dockerfile (multi-stage, prod target)

```
# syntax=docker/dockerfile:1
########## base: the job runner ##########
FROM python:3.12-slim AS base
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 \
    HF_HOME=/work/data/hf \
    SIGNALS_DB_PATH=/work/data/signals.db
WORKDIR /work
RUN apt-get update && apt-get install -y --no-install-recommends \
      ca-certificates curl tini && rm -rf /var/lib/apt/lists/*
COPY . /app
RUN cd /app && pip install -e ".[rerank]"
# pre-bake the pinned MiniLM embedder + reranker (P6: no runtime HF fetch)
RUN python - <<'PY'
from src.retrieval.embedder import get_embedder
from src.signals.rerank import get_reranker
get_embedder(); get_reranker()
PY
COPY docker/entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh && useradd -m -u 10001 signals \
 && mkdir -p /work/data && chown -R signals:signals /work
USER signals
ENTRYPOINT ["/usr/bin/tini", "--", "/entrypoint.sh"]
CMD ["doctor", "--no-network"]
########## browser: adds chromium + optional Xvfb (P4, P8) ##########
FROM base AS browser
USER root
RUN apt-get update && apt-get install -y --no-install-recommends \
      xvfb chromium fonts-liberation && rm -rf /var/lib/apt/lists/*
RUN python -m playwright install chromium && python -m patchright install chromium
USER signals
```

Notes: pytest/respx stay (they are core deps; splitting them is an upstream packaging change — P7); `HF_HOME` redirects the model cache into the data volume; `WORKDIR /work` fixes all CWD-relative state (P1); the antibot engine is intentionally absent (curl_cffi fallback, P10).

Appendix B — Reference docker-compose.yml

```
services:
  signals:
    image: signals:browser-latest
    build: { context: ., target: browser }
    working_dir: /work
    volumes:
      - signals-data:/work/data
```

```
      - ./config/lists:/work/config-lists:ro   # curated lists (operator state)
    env_file: .env                             # never baked into the image
    healthcheck:
      test: ["CMD", "python", "-m", "src.cli", "doctor", "--no-network"]
      interval: 6h
      retries: 1
  scheduler:
    image: signals:browser-latest
    command: ["watch", "--interval-minutes", "60"]
    volumes: [ "signals-data:/work/data" ]
    env_file: .env
    restart: unless-stopped
volumes:
  signals-data:
```

Alternative to the `scheduler` service, per the repo's own recommendation: host cron invoking `docker compose run --rm signals collect` per cadence tier. Pick one; the filesystem lockfile backstops either way.

Appendix C — Kubernetes sketches (for the deferred phase)

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: signals-collect-hourly }
spec:
  schedule: "7 * * * *"
  concurrencyPolicy: Forbid          # single-writer discipline (P3)
  activeDeadlineSeconds: 3600
  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: collect
              image: registry/signals:1.0.0
              args: ["collect"]
              volumeMounts: [ { name: data, mountPath: /work/data } ]
              envFrom: [ secretRef: { name: signals-secrets } ]
          volumes:
            - name: data
              persistentVolumeClaim: { claimName: signals-data }   # ReadWriteOnce
```

Until Postgres (§8.1): every CronJob touching `data/` must serialize (`Forbid` + a single PVC), and nothing may run `replicas > 1`. After Postgres + SQS: swap PVCs for object storage, add a worker Deployment, and the fleet scales freely.

Appendix D — AWS Level 2 reference (private-scale)

```
GitHub Actions ──build/push──> ECR
                                |
EventBridge Scheduler ──invoke──> ECS Fargate RunTask (spot)
   (per source-tier schedules)        |  env from Secrets Manager/SSM
                                      |  signals-data on task store;
```

```
                                  |  DB snapshot -> S3 per run
                                  v
                          S3: raw store / dossiers / backups
                                  |
                          Delivery: SES (digests) + signed webhooks
                          Logs/metrics: CloudWatch; alarms -> email
```

Compute note: for the single-tenant private case, one small EC2 + EBS with this same managed ring around it is equally valid and gives the stable egress IP §5.6 argues for.

Appendix E — Source egress summary (deployment allowlist seed)

Regulatory: `data.sec.gov`, `federalregister.gov`, `usaspending.gov`, DOL WARN, `crt.sh`, `web.archive.org`. ATS: greenhouse, lever, ashby, smartrecruiters, workable, recruitee, workday, rippling, jobvite, breezy, teamtailor + generic careers sitemaps. News: `news.google.com` RSS, blog feeds. Community: HN Algolia, GitHub API, Reddit (off). Marketplaces (opt-in): g2.com, capterra.com, trustradius, softwareadvice, getapp. Identity: wikidata/wikipedia, Google EKG/kgsearch, DDG HTML. X-ray (manual): `lite.duckduckgo.com`/Google. LLM: `openrouter.ai`, `api.typesafe.ai`. CDN: `huggingface.co`. Delivery: SMTP host, webhook URLs. Solvers: `api.2captcha.com`, `api.capsolver.com`.

Appendix F — Assumptions behind the cost figures

All dollar figures are indicative planning ranges (USD, late-2026 knowledge, us-east-1 class) intended for relative comparison, not budgeting; re-verify against current pricing before commitment. Workload assumptions: ~40 vCPU-hours/month of batch tasks; < 1 GB data tree; < 10 GB/month transfer; 1 operator; solver/proxy/LLM metered paths excluded from private scenarios (they are opt-in). VPS prices reflect Hetzner/OVH/DigitalOcean class providers. EKS floor reflects the control-plane fee alone.

Appendix G — Punch list traceability

Item	Where	Evidence
P1 CWD-relative state	`src/core/runlog.py:31`, `src/cli.py:28-40`, `init`	config package-relative: `src/core/config.py:257`
P2 WAL/local-FS	`src/core/db.py:577`, `src/core/filelock.py`	NFS/EFS hazard is structural
P3 in-process rate limits + per-FS single-flight	`src/core/ratelimit.py`, `src/pipeline/watch.py:72-87`	blocks horizontal scale
P4 headed browser paths	`src/antibot/python/ghost.py:62-88`, `src/cli.py:689-692`, `cf_bypass.py:115-216`	Xvfb needed when enabled
P5 sibling repos + Windows venv path	`src/sources/linkedin_db/collector.py:70-110` (path hardcoded at :91), `src/identity/linkedin_ids.py:80-120`	silent no-op if absent

Item	Where	Evidence
P6 HF runtime fetch	`src/retrieval/embedder.py`, `src/signals/rerank.py` (revision-pinned)	pre-bake into image
P7 pytest/respx in core deps	`pyproject.toml:23-24`	split upstream if slimming matters
P8 browsers not auto-installed	no `playwright install` in CI/scripts	Dockerfile must install
P9 no health endpoint	repo-wide grep	net-new with control plane
P10 Windows-only engine build	`src/antibot/README.md`, `BUILD_NOTES.md`	fallback documented, doctor WARN
P11 Windows pid-liveness branch	`src/core/filelock.py:42-52`	POSIX branch exists; CI matrix proves it

End of document. Prepared 2026-10-30 against repo state `f6a8e62`*; file/line references reflect that snapshot.*